

武汉理工大学毕业设计（论文）

Word 文档格式检查与修改软件的设计与实现

学院（系）： 计算机与人工智能学院

专业班级： 计算机科学与技术 zy2101 班

学生姓名： 李国庆

指导教师： 孙玉芬

学位论文原创性声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包括任何其他个人或集体已经发表或撰写的成果作品。本人完全意识到本声明的法律后果由本人承担。

作者签名：

年 月 日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保障、使用学位论文的规定，同意学校保留并向有关学位论文管理部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权省级优秀学士论文评选机构将本学位论文的全部或部分内容编入有关数据进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

本学位论文属于 1、保密口，在 年解密后适用本授权书

2、不保密口。

（请在以上相应方框内打“√”）

作者签名： 年 月 日

导师签名： 年 月 日

摘 要

在信息化办公的场景之下，Microsoft Word 文档格式规范性方面的问题较大，传统依靠手动处理的方式效率不高并且错误率偏高，而现有的工具缺少针对复杂格式的系统性解决办法，本研究借助 python-docx 库开发出 Word 文档格式检查与修改软件，可自动对封面、摘要、正文、参考文献等部分当中字体、字号、段落缩进、引用格式等错误进行检测以及纠正。

研究一开始是借助文献调研来明确文档格式的需求，采用模块化设计去构建系统架构，运用 PyQt6 开发交互界面，达成文件导入、格式预设等功能，文档解析模块依据全连接无环有限状态机模型以及关键词识别算法，精确提取文档结构与格式元数据，格式检查模块借助哈希表存储规则集，将用户预设规范和文档实际格式作对比，找出字体、行距等错误，格式修改模块调用 python-docx 库 API，自动修正检测到的格式问题。经过系统测试，软件可高效处理.docx 文档，支持常见文献引用格式校验，操作流程稳定且可靠。

该软件是 python-docx 库的一次成功运用，可提高学术写作以及办公文档格式处理的效率和准确度，还为同类型的软件开发提供了借鉴。

关键词：Word 文档；格式检查与修改；python-docx 库；自动化处理

Abstract

Under the scenario of information-based office work, there are significant issues with the standardization of Microsoft Word document formats. The traditional manual processing methods are inefficient and have a high error rate, while existing tools lack systematic solutions for complex formats. This study leverages the python-docx library to develop a Word document format checking and modification software that can automatically detect and correct errors in sections such as covers, abstracts, main body, and references, including font styles, sizes, paragraph indents, and citation formats.

Initially, the research began with literature review to clarify the requirements for document formats. A modular design was adopted to construct the system architecture. PyQt6 was used to develop the interactive interface, achieving functions such as file import and format presets. The document parsing module employs a fully connected acyclic finite state machine model and keyword recognition algorithms to precisely extract document structure and format metadata. The format checking module uses a hash table to store rule sets, comparing user-predefined standards with the actual document format to identify errors in fonts, line spacing, etc. The format modification module calls the python-docx library API to automatically correct detected format issues. Through system testing, the software can efficiently process .docx documents, support validation of common citation formats, and ensure stable and reliable operation.

This software represents a successful application of the python-docx library, improving the efficiency and accuracy of academic writing and office document format processing. It also provides a reference for the development of similar software.

Key Words: Word Document; Format Checking and Modification; Python-docx Library; Automated Processing

目 录

第 1 章 绪论	1
1.1 研究背景、目的及意义	1
1.2 国内外研究现状	1
1.2.1 国内研究现状	1
1.2.2 国外研究现状	2
1.3 研究内容、预期目标及结构安排	3
1.3.1 研究内容	3
1.3.2 预期目标与技术路线	4
1.3.3 结构安排	4
第 2 章 系统相关技术介绍	6
2.1 开发技术	6
2.1.1 Python 编程语言	6
2.1.2 python-docx 库	6
2.1.3 PyQt6	6
2.2 开发环境	7
2.2.1 VSCode	7
2.3 本章小结	7
第 3 章 系统需求分析与概要设计	8
3.1 可行性分析	8
3.1.1 技术可行性	8
3.1.2 市场需求可行性	8
3.1.3 经济可行性	8
3.2 用户群体分析	9
3.3 系统功能需求分析	9
3.3.1 文档导入功能	9
3.3.2 用户格式设置功能	9
3.3.3 文档解析功能	9
3.3.4 排版顺序检查功能	10
3.3.5 格式检查功能	10
3.3.6 参考文献引用格式检查功能	10
3.3.7 格式修改功能	10
3.3.8 结果展示功能	10
3.3.9 用户界面模块	10
3.4 数据结构设计	11
3.4.1 格式结构体设计	11
3.4.2 论文部分结构体设计	12
3.5 本章小结	13
第 4 章 系统具体实现	15
4.1 系统架构设计	15
4.2 用户交互界面模块的实现	16

4.3 文档解析模块的实现	18
4.4 格式检查模块的实现	21
4.4.1 格式检查功能	21
4.4.2 文献引用格式检查功能	22
4.4.3 排版顺序检查功能	24
4.5 格式修改模块的实现	26
4.5.1 格式修改功能	26
4.5.2 修改后文档自动保存功能	27
4.6 本章小结	28
第5章 系统测试	29
5.1 测试环境	29
5.1.1 硬件环境	29
5.1.2 软件环境	29
5.1.3 测试方式	29
5.2 系统功能测试	29
5.2.1 文档导入功能测试	29
5.2.2 格式设置功能测试	31
5.2.3 格式检查模块与结果显示功能测试	31
5.3 本章小结	34
第6章 总结与展望	35
6.1 总结	35
6.2 展望	35
参考文献	36
致 谢	37

第 1 章 绪论

1.1 研究背景、目的及意义

随着信息化办公变得日益广泛深入地普及，Microsoft Word 文档已然成为学术研究、企业办公以及日常事务里必不可少的一种载体，不过文档格式的规范性一直都是用户所面临的关键挑战，像学术论文、毕业论文、项目报告这类对格式有着严格要求的文档，就拿高校毕业论文来说，它的格式规范一般包含字体、字号、段落缩进、页眉页脚、参考文献引用等诸多方面，哪怕是很细微的格式错误都有可能致使评审无法依靠甚至需要返工。以往传统的手动检查和修改方式耗费时间和精力，还容易因为人为疏忽而出现漏检或者误判的情况，据相关统计，学生在毕业论文排版方面平均要花费 20 到 30 个小时，并且格式错误率超过 40%，这种效率低下以及错误率高的现象，充分显示出对自动化文档格式处理工具的迫切需求。

近些年来，Python 编程语言凭借其简洁的特性、丰富的生态以及跨平台的优势，逐渐变成办公自动化领域的关键工具，其中 python-docx 库为开发者提供了读取、解析以及修改 Word 文档的接口，极大地降低了文档自动化处理的技术难度，然而现有的研究大多集中在单一功能的实现上，比如文本替换、格式检测或者模板生成等，缺少针对复杂格式问题的系统性解决办法。

在这样的背景状况下，本课题基于 python-docx 库来设计并实现一款 Word 文档自动化格式检查与修改软件，借助解析文档格式特征并纠正错误，以此提升文档处理的效率和规范性，其研究意义体现在实践和理论两个维度：在实践方面，该工具可直接提高学术写作以及办公文档的格式处理效率和精准程度，依靠减少人工操作误差来强化文档质量，同时拓展 python-docx 库的功能范围，推动 Python 在文档处理领域的技术应用，还可以为同类软件开发提供可复用的解决办法参考，在理论方面，研究聚焦于自动化格式检查与修改机制，填补了文档处理领域在格式特征分析与智能纠错方向的理论空白，借助深入挖掘文档 XML 结构中的格式元数据，为文档处理技术的创新提供方法论方面的支持，最终形成一个有标准化流程又有算法模型优化的理论框架，对于推动文档处理自动化理论体系的发展有着关键的价值。

1.2 国内外研究现状

1.2.1 国内研究现状

基于 Python 的办公自动化工具开发旨在通过编程技术提升办公效率，简化文档处理流程。王贞杰和穆静^[1]对 Python 办公自动化类库的分析问题进行了深入探讨，开发了一套基于国产银河麒麟 V10 系统的 Python 类库分析程序，通过树形结构呈现类库中类的关系，

实现了类库代码分析的可视化和智能化。丁烨敏^[2]则从高校毕业设计说明书格式检查的实际需求出发，利用 Python 和 Open XML 技术开发了自动检测系统，有效减少了学生和老师的工作量。罗文兵等^[3]于软件第三方测评电子文档编制工作场景里，设计并实现了一款能在不启动文字处理软件的情况下，批量修改替换 DOCX 文档中文本内容的工具，使得文档编辑效率得到提升，李秀贤^[4]针对 Word 格式的学位论文排版问题展开研究，探讨了基于 Python 的自动排版方法，运用用户数据替换模板范例数据的方式，减少了排版操作，提高了工作效率以及正确率。程睿^[5]在分析国内外研究现状后，提出了基于 Open XML 的 docx 文档创建和修改工具，借助 Open XML 格式规范处理文档格式问题，给出了一种高效且快捷的文档处理办法，各研究者在基于 Python 的办公自动化工具开发领域从不同角度着手，提出了多种解决方案，推动了该领域的持续发展与创新。

基于 Open XML 的文档处理系统是一种运用 Open XML 标准对文档进行高效解析、生成以及管理的先进技术，林晓等^[6]对考试试卷自动生成问题做了分析，提出了一套基于 Open XML 和有限自动机的理论模型，成功达成从大量 WORD 文档中直接抽取题目并排版生成试卷，有效解决了考试保密性与工作效率问题。纳利军^[7]从毕业论文格式规范化角度出发，结合 Open XML SDK 类集和 C# 技术开发了毕业论文格式修订系统，提供了规范化的 Word 模板并实现了全文档自动格式修订，提升了论文格式化的效率和准确性，秦志红^[8]在 DOCX 文档解析及隐藏信息提取方面有关键贡献，提出了基于关联对象和文档扩展空间的信息隐藏检测和提取算法，为电子数据取证技术提供了有力支持。吴为民^[9]依靠研究基于 OXML 结构化格式的 Word 试卷智能处理系统，利用 python-docx 库实现了 Word 文档的增删改查功能，模拟数据库操作，提高了试卷处理的工作效率，姚进林^[10]在基于 OOXML 的 Word 文档服务研究方面取得成果，构建了一个支持并发的在线文档自动生成系统，详细探讨了模板标签设计、文档生成引擎架构、并发处理及实验验证等多方面关键技术，为大型企业和机构的文档生成提供了高效解决方案。基于 Open XML 的文档处理系统在多个领域呈现出强大的应用潜力与实际效果，为文档管理和技术创新提供了关键支撑。

Word 文档解析以及智能处理属于当前文档管理领域里的关键研究方向，借助技术手段提升文档处理的效率以及准确性，陆静^[11]剖析了 Word 文档的多种扩展名及其有的特点，.docx 格式所拥有的优势，刘伟男^[12]针对存在大量相同格式要求的 Word 文档处理问题，依据 OXML 格式规范研发了批量文档智能处理工具，达成了文档的批量格式修改，证实了系统的有效性与实用性。

1.2.2 国外研究现状

在 Word 文档格式检查与修改这个领域，国外的相关研究已经有了一些进展，对于文档生成工具的研究，Machlis^[13]提出了 Quarto 这个技术出版系统，它可支持 Python、Julia

以及 R 语言，还可以生成多种输出格式，像 Word、PowerPoint、HTML 和 PDF 等。Quarto 的优点是有跨平台的兼容性和灵活性，不过它在 Word 格式输出时的样式调整方面以及待优化，在文档特征分析与作者鉴别领域，Asako O. 等学者^[14]提出了创新性的研究方法，借助解析.docx 文档的 XML 结构来获取格式信息，比如字体、缩进、样式应用等元数据，然后结合决策树和随机森林算法构建作者识别模型。这项研究利用大学课程报告数据集验证了文档格式特征对作者身份判别的有效性，为解决教学场景中电子文档抄袭检测的误判问题提供了新的思路，其可视化分类规则的方法也为教师人工复核提供了技术支持，

在 Python 项目的类型相关缺陷研究方面，Khan 等人^[15]凭借实证研究发现，Python 的动态类型系统虽然提供了强大的编程抽象，却也容易致使类型相关缺陷的积累。为了能在早期检测这些缺陷，Python 生态系统中引入了类型注解和静态类型检查器，比如 mypy，然而关于类型检查器在实际项目中的应用效果以及它对缺陷揭示的影响，仍然需要深入探讨，

在文档安全领域，针对 Office OpenXML 格式的 MS-Word 文件，研究人员提出了基于异常结构分析的伪造检测机制^[16]。该研究分析了现有 OOXML-based MS-Word 文件结构的弱点，探讨了数据隐藏和伪造在数字文档中的实现方式，借助内部结构分析，可以有效识别和防范恶意软件和勒索软件的攻击，提升文档的安全性，

国外研究在 Word 文档生成、Python 项目缺陷检测以及文档安全方面都有所涉及，但还是存在一些不足之处。比如 Quarto 在 Word 格式输出时的样式调整问题，Python 类型检查器的实际应用效果评估，以及文档安全机制在复杂环境下的适用性等，这些研究为论文的设计与实现提供了关键的参考和启示，同时也指明了研究的方向。

1.3 研究内容、预期目标及结构安排

1.3.1 研究内容

该研究聚焦于基于 python-docx 库的 Word 文档格式检查与修改软件的设计及实现，首先会针对 Word 文档格式规范展开全面且系统的研究分析，以此明确各类文档格式的具体要求，其次着手设计基于 python-docx 库的文档格式检查算法，保证可精准识别文档里的格式错误。随后实现文档格式自动修改功能，借助算法自动纠正所识别出的格式问题，提高文档的规范性与一致性，同时还会开展软件用户界面的设计与开发工作，保证用户可方便快捷地使用软件的各项功能，最后进行系统的测试以及性能评估，验证软件的实用性与稳定性，依靠对上述研究内容的深入探讨并实现，可解决传统手动检查和修改文档格式时效率低下且容易出错的问题，提升文档处理的自动化水平以及质量。

1.3.2 预期目标与技术路线

预期目标是设计并完成开发的软件能根据论文各部分内容的格式要求检查 Word 文档中相应内容的格式，并对错误格式进行纠正。

需进行检查的文档内容涉及封面、摘要、目录、正文、各级标题、图以及参考文献等，课题的研究方法与技术路线主要含有以下几个环节：首先要开展文献调研，全面系统地查阅并剖析国内外有关 Word 文档格式规范、Python 编程语言及其在文档处理领域应用等文献，以此了解当下研究的前沿动态以及技术发展趋势，这一步能为后续研究工作筑牢理论根基。接着进行需求分析，深入调研用户在 Word 文档格式检查与修改方面的实际需求，明确软件的功能定位和设计目标，然后是系统设计，依据需求分析的结果，设计软件的整体架构以及各个功能模块，主要有文档读取模块、格式检查模块、格式修改模块、用户界面模块等，采用模块化设计理念，保障系统有可扩展性和可维护性。在编码实现阶段，运用 Python 编程语言和 python-docx 库进行软件开发，着重实现文档格式的自动化检查和修改功能，设计高效算法，保证软件的运行效率和准确性，同时开发友好的用户界面，提升用户体验，完成编码后，进入测试与评估阶段，设计多种测试用例，对软件展开全面测试，覆盖功能测试、性能测试、稳定性测试等，借助测试结果评估软件的性能和效果，找出并解决潜在问题。最后进行文档撰写，编写详细的系统设计文档、用户手册等技术文档，保证软件有可使用性和可维护性，同时整理研究成果，撰写研究论文，为后续的研究和推广应用提供参考。

1.3.3 结构安排

本文围绕“Word 文档格式检查与修改软件”的设计与实现展开，共分为 6 章进行详细阐述，各章节内容安排如下：

第 1 章绪论，会介绍一下研究背景，以及国内外研究的现状情况，阐明研究的目的、意义以及核心内容，以此明确论文整体的框架和逻辑结构。^①

第 2 章对系统相关技术给予介绍，剖析软件开发过程中所采用的技术和工具，覆盖 Python 编程语言、python-docx 库、PyQt6 框架以及 VSCode 开发环境各自的特点与应用场景。^②

第 3 章进行系统需求分析与概要设计，开展可行性分析以及用户群体调研工作，梳理功能需求并设计数据结构，借助用户用例图和结构体设计来明确系统边界与实现方向，

第 4 章是系统具体实现，会详细描述软件各个功能模块的技术实现细节，比如系统架构、用户界面开发、文档解析算法、格式检查与修改逻辑等方面。

第 5 章为系统测试，说明测试环境与方法，凭借具体测试用例来验证文档导入、格式设置、检查、修改等核心功能的稳定性和准确性，

第 6 章是总结与展望，总结研究成果与软件性能，提出技术升级、应用拓展以及用户体验优化的方向，展望未来的发展前景。

第 2 章 系统相关技术介绍

2.1 开发技术

2.1.1 Python 编程语言

Python 作为一种高级编程语言，在当下软件开发领域有着广泛应用，它为本次 Word 文档格式检查与修改软件的开发给予了关键技术支持，Python 有简洁且易读的语法，这让开发人员可更高效地编写代码并理解代码内容，减少了因语法复杂致使的开发时间增长以及出错概率。比如在处理复杂的文档格式检查逻辑时，Python 简洁的代码结构可使开发人员更清晰地梳理思路，迅速实现功能，

Python 拥有丰富的第三方库生态系统，对于本软件来说，python-docx 库是核心依赖之一，借助这个库，开发人员可轻松地对 Word 文档进行读取、解析以及修改操作。凭借简单的代码调用，便能获取文档中的文本内容、格式信息，并对这些信息进行分析与调整，极大程度降低了文档处理的难度，

Python 还有跨平台特性，不管是 Windows、Mac 还是 Linux 系统，基于 Python 开发的软件可稳定运行，这为软件的广泛应用给予了便利，不同操作系统的用户都可以使用本软件进行 Word 文档格式的检查与修改，不用担心系统兼容性问题。

2.1.2 python-docx 库

python-docx 库作为 Python 第三方库，专门用于处理 Word 文档，在本软件的开发里有着不可替代的作用，它给出了直观且好用的 API 接口，开发人员可借助这些接口直接访问 Word 文档的各个元素，像可迅速定位文档里的各级标题、正文段落、图片、表格以及参考文献等内容，还可以对其格式给予读取和修改。

这个库支持对文档样式进行操作，开发人员可以依据预先设定好的格式规范，借助代码修改文档中元素的样式，像字体样式、段落缩进、行距等，此功能对实现文档格式的标准化处理十分关键，保证了软件在检查和修改文档格式时可精准地依照要求操作，提升了文档处理的规范性与一致性。

python-docx 库还具有良好的扩展性，随着软件功能需求的增多，比如未来要支持更多复杂的文档格式处理功能，开发人员可以基于该库开发与拓展，灵活地契合不同用户的多样需求。

2.1.3 PyQt6

PyQt6 是一款功能颇为强大的 Python GUI 框架，它是基于 Qt 库来进行开发的，在本软件里承担着构建直观且易用的图形用户界面的任务，PyQt6 提供了种类丰富的 UI 组件，像按钮、文本框、列表框之类的，开发人员借助简单的代码组合，就能快速搭建出符合用

户操作需求的界面。举例来说，在软件设计文件选择界面、格式设置界面以及结果展示界面的时候，借助 PyQt6 的组件可高效达成布局设计，

它支持信号与槽机制，依靠这种机制能让软件的交互逻辑实现得更为便捷，当用户在界面上开展操作，比如点击“开始检查”按钮时，借助信号与槽机制，可以顺利连接到对应的文档格式检查函数，达成用户操作与后台功能的无缝对接，提升用户体验。

PyQt6 同样拥有跨平台特性，保证了软件在不同操作系统下界面风格与功能的一致性，不管是 Windows、Mac 还是 Linux 系统的用户，都可获得质量相同的使用体验，它与 Python 紧密结合，在 VSCode 开发环境中，可和 python-docx 库等其他 Python 库协同工作，共同完成文档格式检查与修改软件的各项功能。

2.2 开发环境

2.2.1 VSCode

VSCode 是一款有轻量级特质同时功能颇为强大的源代码编辑器，其具有较高的可定制性，十分契合本 Word 文档格式检查与修改软件开发工作，它拥有智能化的代码编辑功能，对 Python 代码提供语法高亮显示支持、智能代码补全以及代码片段插入功能，在编写软件代码期间，可极大程度提升编码效率。

在调试层面，VSCode 集成了功能强大的调试工具，开发人员可便捷地设置断点，逐步开展代码调试工作，实时查看变量值以及程序执行流程，针对软件里复杂的文档格式检查与修改逻辑，以及借助 PyQt6 实现的界面交互逻辑，借助这些调试工具，可迅速定位并解决问题，保障软件的稳定性与可靠性。

VSCode 的插件市场资源较为丰富，开发人员可借助安装插件来扩展编辑器功能，在开发本软件进程中，可安装 Python 插件用以提高对 Python 语言的支持，安装相关代码格式化插件，让代码更规范且易于阅读，借助插件还可以便利地管理 Python 项目依赖，对于 python-docx 库以及 PyQt6 的安装与版本管理均可轻松达成，提升开发效率。

2.3 本章小结

在这一章节之中，会对 Word 文档格式检查与修改软件所运用的开发技术以及开发环境给予详细介绍，Python 编程语言有简洁的特性、丰富的库资源以及跨平台的特点，这为该软件的开发提供了有力的支撑，python-docx 库专门希望能够 Word 文档处理，可提供高效的文档操作接口。PyQt6 则是用于构建优质的图形用户界面，以此实现良好的用户交互，VSCode 开发环境拥有强大的代码编辑、调试功能以及丰富的插件扩展能力，提升了软件开发的效率与质量，这些技术以及环境相互协作配合，为软件的成功开发奠定了坚实的基础。

第3章 系统需求分析与概要设计

开展 Word 文档格式检查与修改软件的开发工作前，系统需求分析以及概要设计属于极为关键的环节，借助充分的分析以及合理的设计，可保证软件契合用户实际需求，拥有不错的性能以及可扩展性，为后续开发工作筑牢根基。

3.1 可行性分析

可行性分析在项目搭建进程里占据着关键位置，主要是针对项目于技术、市场需求以及经济等层面的可行性状况加以评估，此章节会对 Word 文档格式检查与修改软件的开发展开可行性分析，借此提前识别并且预防有可能出现的问题，借助这一分析，可更为高效地应对各种挑战，保障项目顺利推进。

3.1.1 技术可行性

Python 是一种成熟且应用广泛的编程语言，有着丰富的第三方库资源，其中 python-docx 库专门用于 Word 文档处理，提供了读取、解析以及修改文档的便捷接口，这使得开发者可轻松获取并操作文档中的各类元素，像文本、格式、段落等信息，在开发环境选择方面，以 VSCode 为例，它集成了智能代码编辑、调试、版本控制等诸多强大功能，同时还拥有丰富的插件生态，可契合不同的开发需求。借助 VSCode 的 Python 插件，可以实现代码语法检查、智能补全，提高开发效率，借助安装相关调试插件，能更便利地对软件进行调试，定位并解决问题，结合 Python 和 python-docx 库的技术优势，以及 VSCode 良好的开发支持，本软件在技术实现上有较高的可行性。

3.1.2 市场需求可行性

在学术研究以及企业办公等诸多领域当中，对于 Word 文档格式规范方面的要求是非常高的，就拿学术论文来说吧，要是格式出现错误的话，那么很有可能会对评审结果产生影响，增加作者的时间成本，而在企业里，如果项目报告的格式不统一，就会使得文档的专业性以及可读性有所降低。

当下，虽说部分办公软件自身带有简单的格式检查功能，然而针对复杂的格式规范，像是特定学术期刊或者企业内部的格式要求，却没办法精准地契合需求，手动去检查和修改效率很低，而且还容易出现疏漏，开发一款专业的 Word 文档格式检查与修改软件，可契合市场对于高效且精准的文档格式处理工具的迫切需要，有着广阔的市场前景。

3.1.3 经济可行性

Python 是开源的编程语言，可免费使用和分发。python-docx 库同样遵循开源协议，开发者无需支付额外费用即可在项目中使用。VSCode 作为一款跨平台的免费开源开发工具，

其丰富的功能足以满足本软件的开发需求，无需购买昂贵的商业开发环境。

开发过程中，若涉及到一些辅助工具或服务，如代码格式化工具、代码分析工具等，也有众多免费或低成本的开源替代品可供选择。因此，从经济角度看，本软件的开发成本较低，具备经济可行性。

3.2 用户群体分析

清楚知晓软件的目标用户群体以及他们的需求，对于设计出更符合用户使用习惯和期望的产品是有帮助的，本软件的主要用户群体覆盖学术研究者、企业办公人员以及学生群体，学术研究者在撰写论文、研究报告的时候，要遵循严格的学术格式规范，对文档格式的准确性和规范性有着很高的要求，他们期望软件可精确检查并纠正格式错误，同时支持多种学术期刊的格式标准。企业办公人员在处理各类商务文档时，追求文档格式的一致性与专业性，方便文档的共享、阅读以及存档，他们希望软件操作简单，可迅速处理大量文档，提升工作效率，学生群体在撰写课程作业、毕业论文时，也大多时候会因为文档格式问题花费大量时间和精力，他们需要软件拥有简洁易懂的操作界面，可依照学校或专业的格式要求进行检查和修改，帮助他们顺利完成学业任务。

3.3 系统功能需求分析

本软件根据用户操作流程和功能模块，可分为文档导入、用户格式设置、文档解析、格式检查、文献引用格式检查、格式修改、结果展示和用户界面等功能模块。

3.3.1 文档导入功能

支持多种方式导入 Word 文档，如通过文件选择对话框选取本地文档、从常用文件夹快捷导入，或直接将文档拖曳至软件界面进行导入。导入过程中，能够自动识别文档格式，若遇到损坏或不兼容的文档，给出相应提示信息，引导用户进行处理。

3.3.2 用户格式设置功能

用户可通过该功能按论文部分自定义格式规范，包括对字体、字号、对齐方式、行距等内容进行设置，作为格式检查与格式修改功能的依据。

3.3.3 文档解析功能

能够快速准确地读取 Word 文档，支持.docx 格式。在读取过程中，将文档内容按封面、摘要、目录、正文、各级标题、图、表、参考文献等部分进行解析，保留文档的所有内容和格式信息，为后续的格式检查和修改提供完整的数据基础。

3.3.4 排版顺序检查功能

该功能主要负责检查 Word 文档的排版顺序，判断是否存在部分缺失或顺序错误的情况。在检查过程中，会依据标准的论文结构顺序，如封面、摘要、目录、正文、参考文献、致谢等，对文档进行逐部分检查。对于每个部分，会先确认其是否存在，若存在则检查其位置是否符合标准顺序。一旦发现问题，将精准定位错误位置，并给出详细的错误提示信息，如“缺少摘要部分”或“致谢部分应位于参考文献之后，当前位置错误”等。

3.3.5 格式检查功能

涵盖全面的格式检查项目，包括但不限于字体、字号、段落格式（行距、对齐方式）、参考文献引用格式等。检查文档当前格式与用户设置的目标格式是否一致，并返回一定格式的检查结论以便于检查结果展示。

3.3.6 参考文献引用格式检查功能

专门针对文档中的参考文献引用部分进行检查，支持常见的引用格式规范，如 APA、MLA、GB/T 7714 等。通过匹配引用格式规则，检查引用序号的格式、位置，参考文献列表的排版格式，以及引用内容与参考文献列表的对应关系等。若发现错误，同样精准定位错误位置，并提供详细的错误提示，辅助用户快速修正参考文献引用格式问题。

3.3.7 格式修改功能

针对格式检查模块发现的错误，软件可自动进行格式修改。用户也可选择按照错误提示进行手动修改。修改完成后，软件将自动生成新的 Word 文档，并按照“formatted_原文件名”的命名规则保存，不会对原文件造成任何影响，确保用户原始文档数据的完整性。

3.3.8 结果展示功能

以直观清晰的界面展示格式检查结果，通过列表形式呈现所有格式错误信息，包括错误位置、错误类型等。

3.3.9 用户界面模块

设计简洁友好、易于操作的用户界面。用户可以通过界面方便地导入文档、进行格式设置、启动格式检查、排版顺序检查和修改功能、查看错误提示和修改结果。

该软件的用户用例图如图 3.1 所示：

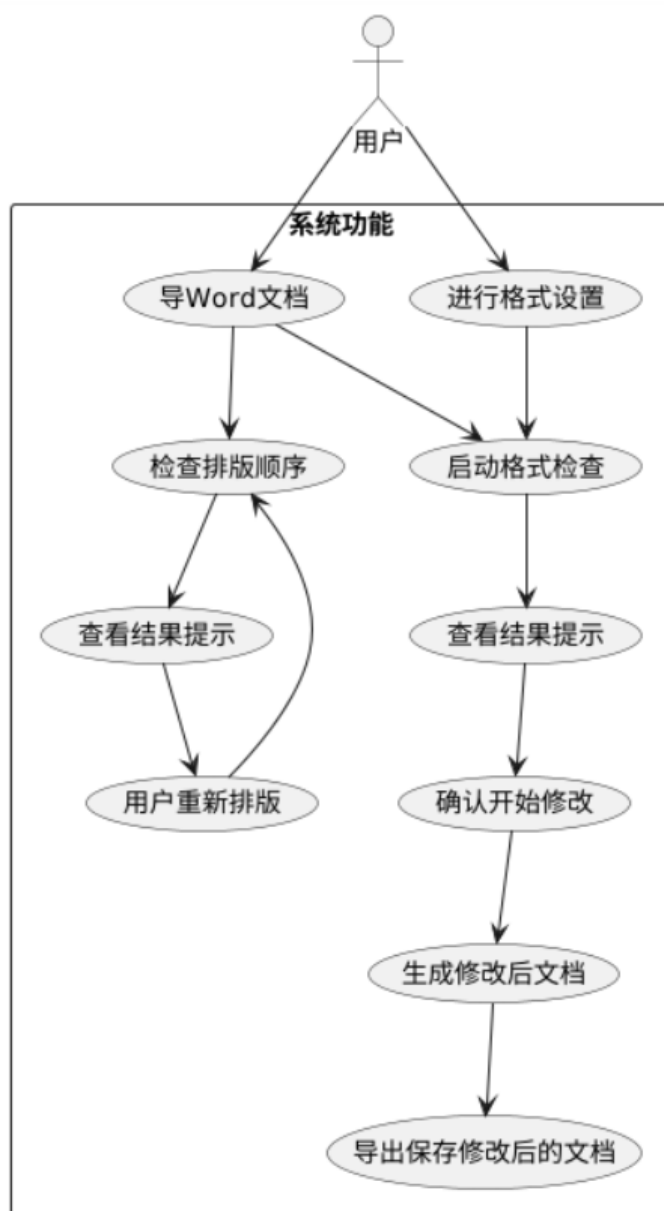


图 3.1 用户用例图

3.4 数据结构设计

在 Word 文档格式检查与修改软件中，数据结构的设计对于高效处理文档格式信息至关重要。它直接关系到软件对文档各部分格式的解析、存储和修改操作的便捷性与准确性。以下主要介绍格式结构体和论文部分结构体的设计。

3.4.1 格式结构体设计

格式结构体用于存储和管理文档各部分的格式信息，它是软件进行格式检查和修改的核心数据结构之一。格式结构体的设计如表 3.1 所示：

表 3.1 格式结构体

结构体成员	数据类型	说明
字体（font）	字符串	记录文档元素的字体，取值来自字体映射表，如“华文中宋”“黑体”等，软件通过字体映射将其转换为实际可操作的字体设置
字号（size）	浮点数	以 pt 为单位表示字号大小，取值从字号映射表中转换而来，如“一号(26pt)”会转换为 26.0，用于精确设置文档元素的字号
对齐方式（alignment）	整数	对应对齐方式映射表中的值，如“居中”对应 WD_ALIGN_PARAGRAPH.CENTER，软件依据此值对文档元素进行对齐操作
行距规则 （line_spacing_rule）	整数	对应行距规则映射表中的值，例如“3 倍行距”对应 WD_LINE_SPACING.MULTIPLE，用于确定行距的计算方式
行距（line_spacing）	浮点数	表示具体的行距数值，对应行距映射表中的值，如“3 倍行距”对应 3.0，结合行距规则确定文档元素的实际行距

这种设计使得格式信息的存储和调用更加清晰、有序，方便软件根据不同的格式需求对文档进行处理。

3.4.2 论文部分结构体设计

论文部分结构体用于组织和管理论文不同部分的内容及对应的格式信息，有助于软件对论文整体结构的把握和处理。论文部分结构体的设计如下表 3.2 所示：

表 3.2 论文部分结构体

论文部分	结构体内容
封面（cover）	包含封面标题、封面相关内容（如学校、论文

论文部分	结构体内容
	题目、个人信息等）及封面格式设置
声明（statement）	包含声明标题、声明内容及声明格式设置
摘要和关键词（abstract_keyword）	包含中文摘要标题、中文摘要内容、中文关键词标题、中文关键词、英文摘要标题、英文摘要内容、英文关键词标题、英文关键词及相应格式设置
目录（catalogue）	包含目录标题、目录内容及目录格式设置
正文（main_text）	包含正文内容及正文格式设置
各级标题（headings）	包含各章标题、一级标题、二级标题、三级标题的标题、内容及格式设置
图或表题（figures_or_tables_title）	包含图或表的标题内容及格式设置
图（figures）	包含图的相关信息及格式设置（若有）
表（tables）	包含表的内容及格式设置
参考文献（references）	包含参考文献标题、参考文献内容及参考文献格式设置
致谢（acknowledgments）	包含致谢标题、致谢内容及致谢格式设置

通过这样的论文部分结构体设计，软件能够方便地对论文各部分进行单独处理，同时也便于整体管理和维护论文的结构与格式。

3.5 本章小结

本章围绕 Word 文档格式检查与修改软件展开，从多个关键角度进行了分析与设计，在可行性分析阶段，从技术、市场需求以及经济这三个维度详细论证了项目的可行性，技术层面，Python 语言及其丰富的库为开发给予了有力的支持，市场方面，对高效文档格式处理工具的需求十分迫切，经济上，开源资源的利用较大降低了开发成本，为项目的开展

奠定了坚实的基础。

针对用户群体，明确了高校学生、科研人员以及企业办公人员等作为主要用户，深入剖析了他们在文档格式处理方面的需求与痛点，保证软件功能设计可精准契合用户期望，

系统功能需求分析部分，详细阐述了软件的各个功能模块，涉及用户格式设置、文档解析、格式检查、参考文献引用格式检查、格式修改以及用户界面。清晰界定了每个模块的功能职责，并且凭借用例图直观展示了用户与软件的交互流程，让软件功能架构一目了然，

数据结构设计取代传统数据库设计，提出格式结构体和论文部分结构体的设计方案，格式结构体精确包含字体、字号、对齐方式、行距等关键格式信息，论文部分结构体则有效整合了论文各部分的标题、内容以及格式，为软件高效处理文档格式提供了合理的数据组织方式。

这些工作共同为 Word 文档格式检查与修改软件的后续开发搭建了完整的框架，为实现高效、准确的文档格式处理功能提供了明确的方向与坚实的技术支撑。

第 4 章 系统具体实现

此章节聚焦于 Word 文档格式检查与修改软件的核心功能，会细致地讲述其具体的实现思路与办法，该软件以 Python 语言为基础，依靠 python-docx 库来开展开发工作，借助这个库所提供的接口达成对 Word 文档的读取、解析以及修改操作，整体运用模块化设计，主要覆盖文档解析、格式检查、格式修改以及用户界面交互等功能模块，各个模块相互协作，以此实现高效且精准的文档格式处理目的。

4.1 系统架构设计

在这部分内容里会具体讲述 Word 文档格式检查与修改软件的系统架构。软件基于模块化设计原则，将核心功能拆解为交互界面模块、文档解析、检查与修改模块四大模块，通过松耦合架构提升系统可维护性。此软件可有效处理和文档格式有关的任务，它的架构设计把交互界面、文档解析、格式检查以及修改等多个关键模块融合到了一起，就像图 4.1 展示的那样，软件借助交互界面达成用户和系统之间的交互，涉及文件导入、格式预设等操作。文档解析器会对文档的各个组成部分展开剖析，格式检查器依照预设规则开展细致检查，格式修改器在确认之后会对文档实施精准修改，这样的架构设计保障了软件在文档格式处理方面的高效性以及准确性。

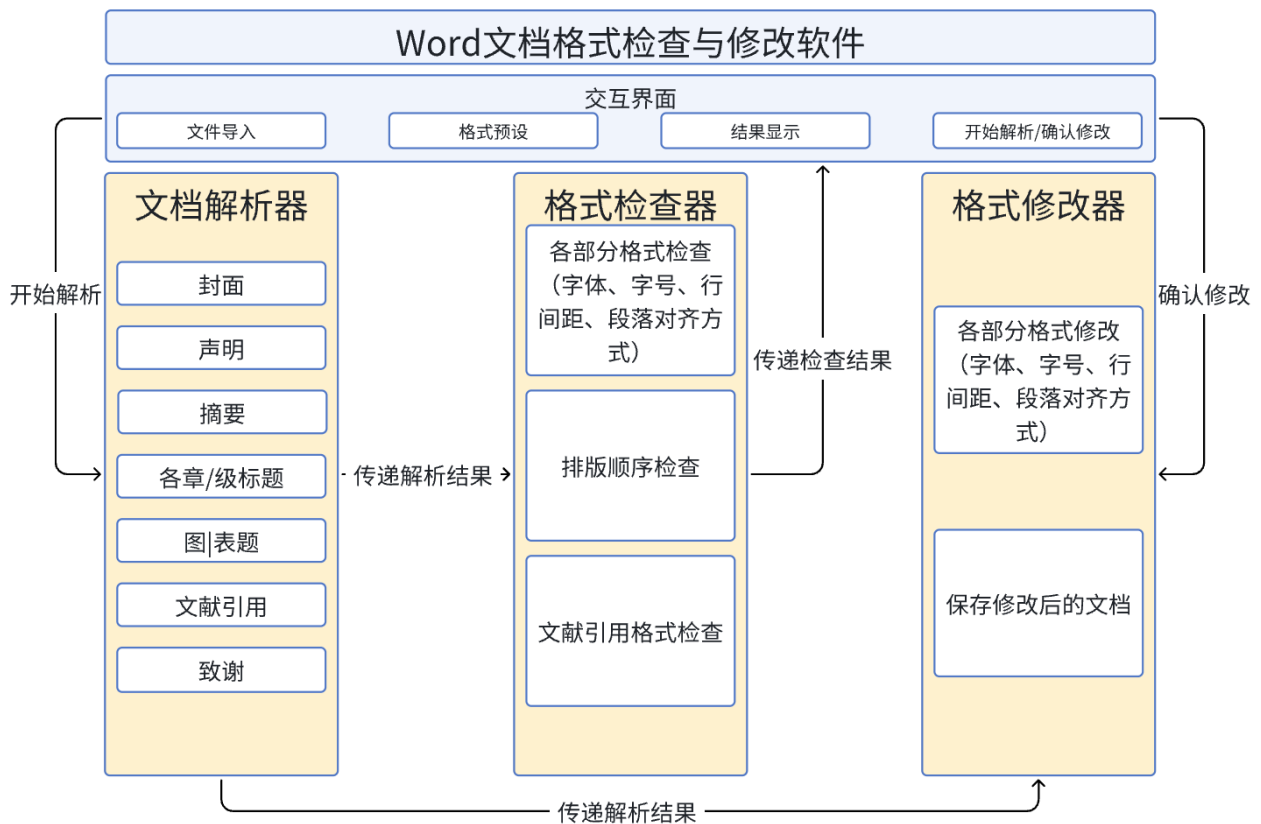


图 4.1 软件系统架构图

4.2 用户交互界面模块的实现

在实现用户交互逻辑时，借助 PyQt6 的信号与槽机制实现了各个组件之间的联动与响应。同时，为了提升用户体验，对界面元素进行了合理的视觉设计和交互反馈处理。例如，当用户将鼠标悬停在按钮上时，按钮会改变颜色以提示可点击；操作成功或失败时，会弹出相应的提示框告知用户。用户交互界面模块通过精心的设计与实现，为用户提供了一个友好、易用的操作环境，使得文档格式检查与修改操作更加流畅和高效。

用户交互模块为用户打造了直观且便捷的操作界面，使用户可顺利完成文档的选择、格式检查修改以及排版顺序检查等一系列操作，该模块运用 PyQt6 库进行开发，其界面提供了直观的交互方式，像是文件拖拽、依靠下拉菜单选择格式等，实现了良好的人机交互体验，界面布局被划分成多个功能区域：

$$F = ma \quad (4.1)$$

文件操作区提供了文件选择按钮以及支持文件拖拽的交互方式，文件选择按钮与系统文件选择对话框相关联，用户点击此按钮后可在本地文件系统中查找并选择 Word 文档。文件拖拽功能是依靠重载 `QWidget.dropEvent` 事件得以实现，并利用 `QUrl.toLocalFile()` 获取文件路径，同时通过 `QFileDialog.setNameFilter("Word Documents (*.docx)")` 来过滤文件类型。当检测到合法的 Word 文档被拖拽至指定区域并释放时，系统会自动获取文件路径并开展后续处理，这大大提升了文件导入的便捷程度。如图 4.2 所示，选择成功后，文件路径会显示在该区域上。

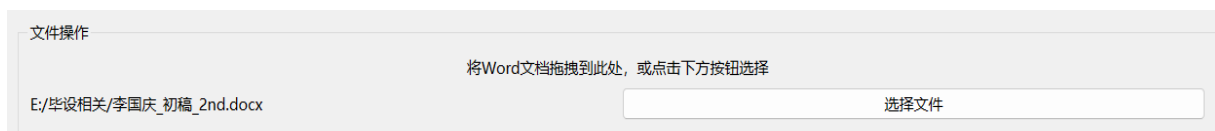


图 4.2 文件操作区

格式设置区针对不同的文档部分，如封面、摘要、正文等，利用 `QTabWidget` 设置了不同的 tab 页，并利用 `QWidget` 分别设定了字体、字号、对齐方式、行间距等格式选项，这些选项借助 `QComboBox` 实现以下拉菜单形式呈现。如图 4.3 所示：



图 4.3 格式设置区

操作按钮区包含“选择文件”“检查格式”“检查排版顺序”“确认修改”等关键按钮，每个按钮都借助信号与槽机制的 connect 方法将 QPushButton 的 clicked 信号与相应的功能函数绑定在一起。“检查格式”按钮被点击后，会发出信号触发用户界面的 check_format 方法进入格式检查模块的执行，并等待其返回检查结果；“检查排版顺序”按钮被点击后则是会触发用户界面的 check_order 方法进入检查排版顺序的处理；“修改格式”按钮则在用户确认检查结果后，点击触发用户界面的 modify_format 方法，启动格式修改模块对文档进行处理。该按钮区如图 4.4 所示：

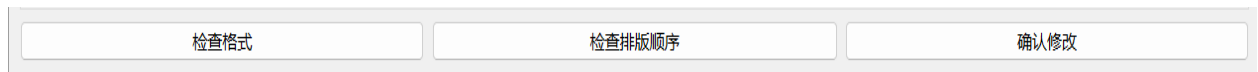


图 4.4 操作按钮区

检查结果区以表格或列表的形式展示格式检查过程中发现的错误信息，表格列包含错误位置、错误类型、错误描述等字段，当格式检查模块返回错误报告后，系统会遍历报告数据，并利用 QTableWidgetItem 动态地将每条错误信息按照对应字段插入到对应表格中，并通过 QMessageBox 弹出排版顺序错误提示，方便用户清楚查看与定位修改。该部分的界面如图 4.5 所示：

$$E = mc^2 \quad (4.2)$$



图 4.5 检查结果区

在实现用户交互逻辑时，借助 PyQt6 的信号与槽机制实现了各个组件之间的联动与响应，为了提升用户体验，对界面元素进行了合理的视觉设计和交互反馈处理，比如当用户将鼠标悬停在按钮上时，按钮会改变颜色以提示可点击，操作成功或失败时，会弹出相应的提示框告知用户。用户交互界面模块借助精心设计与实现，为用户提供了一个友好且易

用的操作环境，让文档格式检查与修改操作更加流畅高效。用户交互界面的整体设计如图 4.6 所示：

$$Output = LayerNorm(x + Sublayer(x)) \quad (4.3)$$

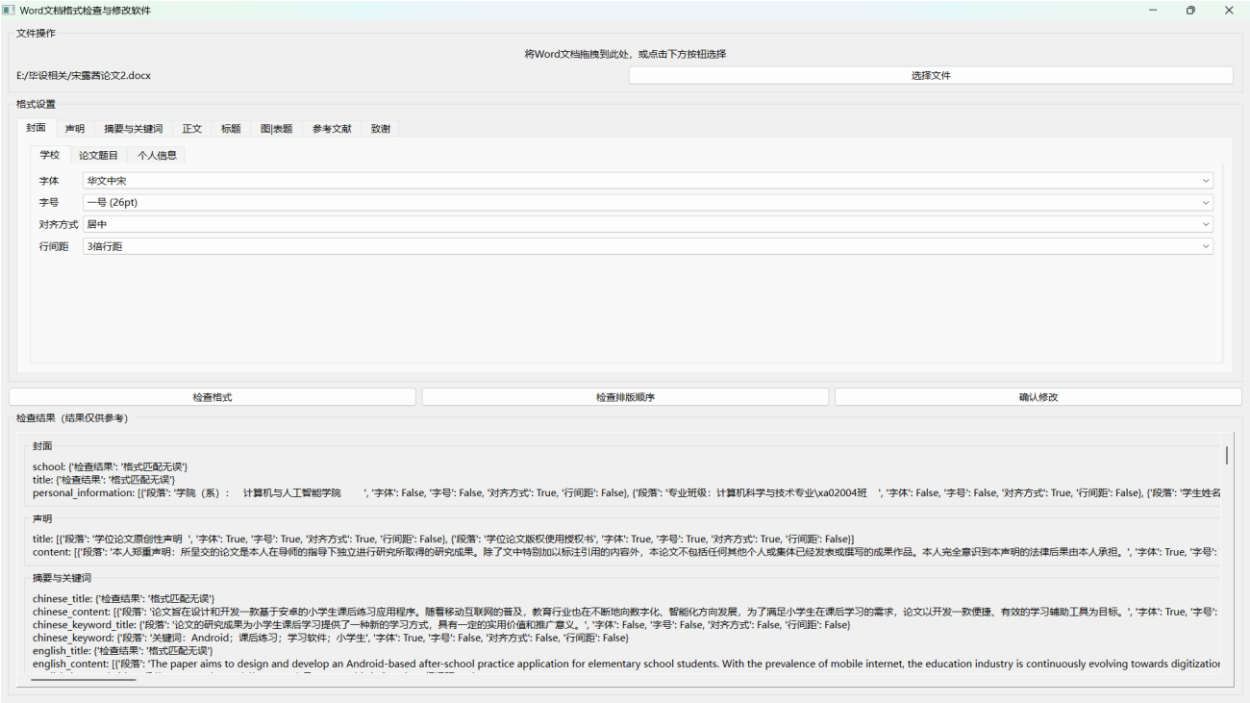


图 4.6 用户界面

4.3 文档解析模块的实现

文档解析模块基于 python-docx 库进行实现，负责读取 Word 文档并提取其结构和格式信息，为后续的格式检查和修改提供基础数据。

核心功能是在遍历文档段落的同时，解析文档的各部分、表格、图片等元素；提取文本内容及对应的格式信息；识别文档不同部分及其结构。该模块会先识别当前遍历的位置是文档的哪一部分，再根据所处部分进行不同的解析动作。关键解析步骤包括文档结构划分、文档元素遍历与识别、格式信息提取。

文档结构划分：该功能的算法实现是采用全连接无环有限状态机模型搭配关键词识别算法来确定文档的不同部分（封面、摘要、正文、参考文献等），定义包含 START、COVER、ABSTRACT、MAIN_TEXT、END 等论文部分与开始和结束的状态集合。确保封面、摘要、正文等部分都能识别到且不会重复解析，同时也不会因为排版顺序而无法识别。提高了软件的适用范围与解析稳定性。在遍历段落时，首先进入“开始遍历段落”状态，当识别到特定关键词（如“摘要”“关键词”“正文”等）时，触发状态转移，解析该部分内容，并从状态表中移除当前已识别到的部分，防止重复解析。每部分都会有专门对应的解析方法，具体会在文档元素遍历与识别中进行举例说明。若所有段落遍历结束或所有部分均已分析

完成，则进入“遍历结束”状态。

该模型的状态序列如图 4.7 所示。

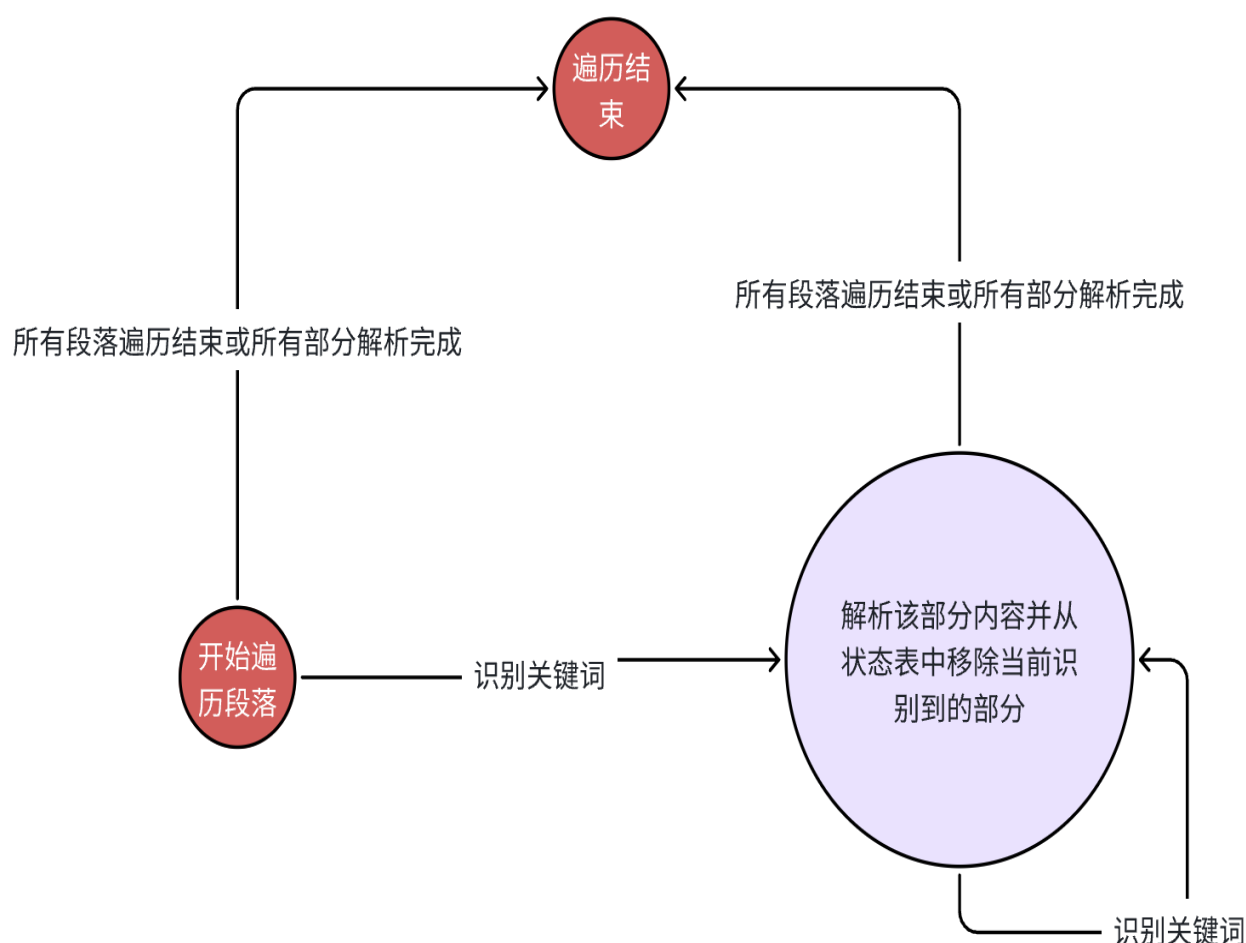


图 4.7 文档解析状态机状态序列

文档元素遍历与识别：解析过程中，通过遍历 `doc.paragraphs` 获取段落，利用 `python-docx` 库提供的 API 获取每个元素的相关属性。利用 `paragraph.font.name` 获取字体；利用 `paragraph.font.size` 获取字体大小，其对应的单位是 `pt`；利用 `paragraph.paragraph_format.line_spacing` 获取段落行间距。

在文档元素的识别中，每一部分有专门的解析方法。例如，在识别到“第 1 章”时，会进入到正文解析方法 `parse_cover`，而对于接下来遍历到的正文段落元素，获取其文本内容，并利用 `python` 的 `re` 模块的正则式匹配，判断当前段落是章标题、节标题、图题、表题或是正文内容，去存入到对应的字典位置，方便检查模块进行取用，如若识别到另一部分的关键词，则进入下一部分的解析方法。

对于中文摘要、英文摘要、致谢等部分，则会将首段作为该部分的标题，余下部分为该部分内容。而对于文献引用，除了标题与内容的拆分以外，还会按序号逐一拆解文献条

目。

正文部分的解析算法如图 4.8 所示：

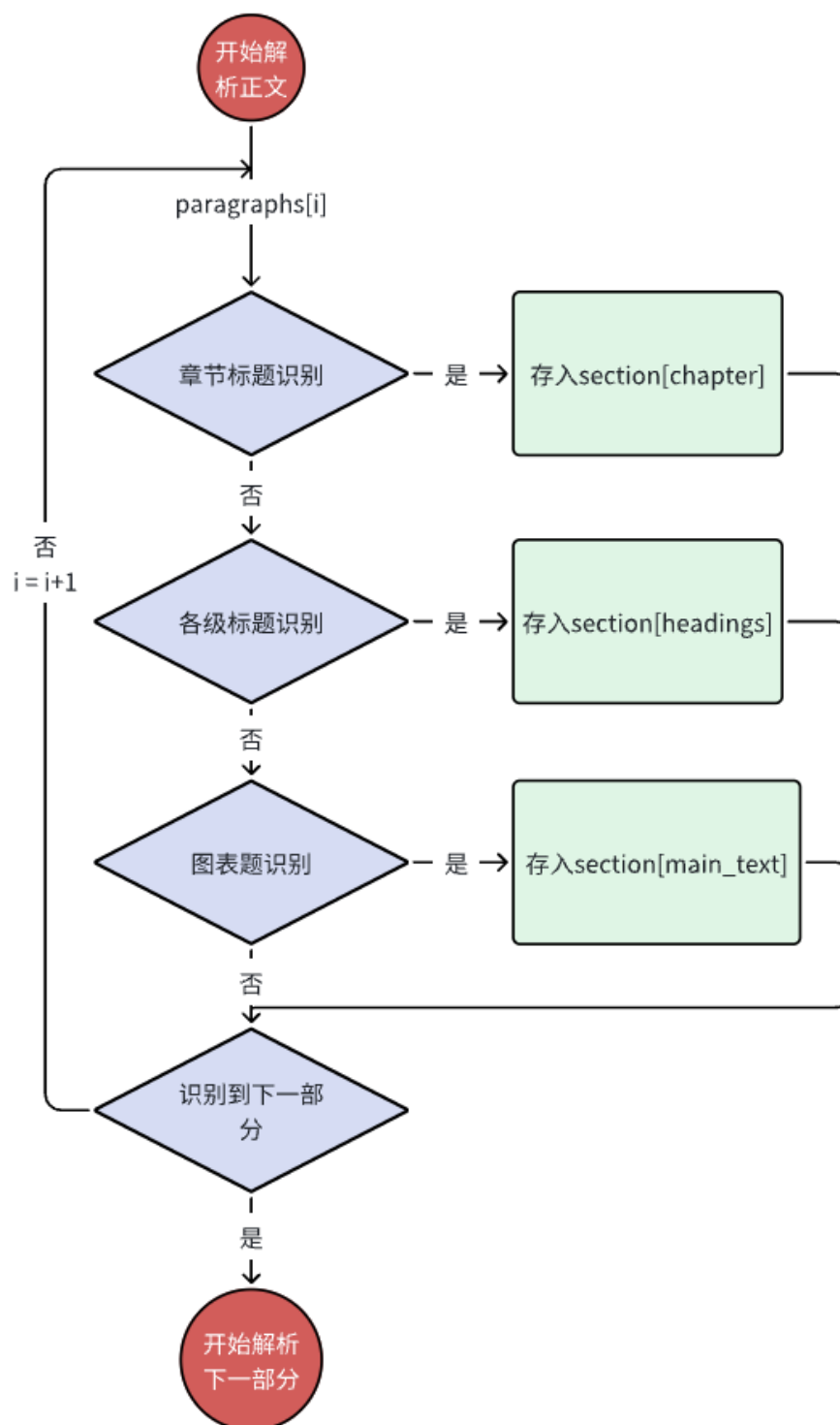


图 4.8 文档解析状态机状态序列

格式信息提取：针对不同的文档元素，分别提取其格式信息。对于段落中的文本，获

取其字体、字号等格式属性；对于段落本身，获取其行距、对齐方式等布局属性。将这些格式信息按部分整理成结构化的数据，便于后续模块使用。

4.4 格式检查模块的实现

格式检查模块负责依据用户预设的格式规范，利用 python-docx 库的相关能力对文档解析模块提取出的文档结构和格式信息进行校验，并对文献引用部分的文献引用格式进行单独检查，找出其中不符合规范的部分，生成检查结果传递给用户界面模块，为后续的修改操作提供明确的问题清单。核心功能有格式检查、文献引用格式检查与排版顺序检查三大功能。

4.4.1 格式检查功能

格式检查功能依据用户事先自行定义的规则集合来达成，这些规则包含了字体、字号、对齐方式、行间距等多种格式属性的标准设定，其核心思路是在收到文档解析数据后，把文档各个部分的实际格式和对应规范逐个进行比对，精准找出格式错误，该功能主要是针对文档里各类文本元素的格式属性做细致校验，保证其符合预先设定的规范标准。

在实现算法方面，运用规则-匹配模式，针对每一种文档元素类型，都有一套详细的规则集与之相对应，凭借对划分好的文档各部分元素进行遍历，将解析时利用 paragraph 的相关 API 获取的字体、字号、对齐方式和行间距等实际格式属性和规则集中预设的标准属性做对比判断，主要的检查项目有字体字号检查、段落格式检查。

如图 4.9 所示，封面-学校预设格式为“华文中宋-一号-居中-3 倍行距”，经过一一比对，检查结果显示字体不符，而字号、对齐方式、行间距均符合预设标准格式。

格式设置

封面 声明 摘要与关键词 正文 标题 图|表题 参考文献 致谢

学校 论文题目 个人信息

字体 华文中宋

字号 一号 (26pt)

对齐方式 居中

行间距 3倍行距

检查格式 检查排版顺序 确认修改

检查结果 (结果仅供参考)

封面

school: {'段落': '武汉理工大学毕业设计 (论文)', '字体': False, '字号': True, '对齐方式': True, '行间距': True}

title: {'检查结果': '格式匹配无误'}

personal_information: [{'段落': '学院 (系): 计算机与人工智能学院', '字体': False, '字号': True, '对齐方式': False, '行间距': False}, {'段落': '专业班级: 计算机科学与

图 4.9 格式检查示例

字体字号检查：对文档中的所有段落和文本区域进行遍历，把当前字体、字号与规范要求做对比。比如规定封面标题字体应为华文中宋、字号二号，正文部分字体为宋体、字号小四等。一旦发现实际字体或字号与之不符，便记录错误信息，包括错误出现的具体位置（如第几页第几段）、错误类型（字体错误或字号错误）以及详细描述（如实际字体为黑体，期望字体为华文中宋）。

段落格式检查：检查段落的对齐方式（左对齐、居中对齐、两端对齐等）、行间距（单倍行距、1.5 倍行距等）。以学术论文为例，正文段落通常要求两端对齐、1.5 倍行距，若实际文档不满足这些要求，就会在检查结果中体现。如图 4.10 所示：



图 4.10 检查结果示例

4.4.2 文献引用格式检查功能

文献引用格式检查功能主要希望能够保证文档中文献引用准确且规范，让引用契合特定的学术及行业标准，该功能会针对文档末尾的参考文献列表开展引用格式校验工作，这其中涉及了文献条目的排版格式，像作者姓名、文献标题、期刊名称、发表年份、卷号、页码等信息的先后顺序以及分隔方式等内容。不同的学术规范，例如 APA、MLA、GB/T 7714 等，对参考文献格式有着不同要求，此模块会依据默认的规范标准来进行检查，在实现的时候，依靠编写专门的文本解析函数来处理引用标注以及参考文献列表的文本内容，借助正则表达式等技术手段提取关键信息并进行格式匹配与一致性验证，把发现的错误信息整理到错误报告里。如图 4.11 所示，示例中[10]和[12]两篇文献引用格式有误，用户可对照标准进行比对修改。这里以武汉理工大学的规范为例，如图 4.12 所示。

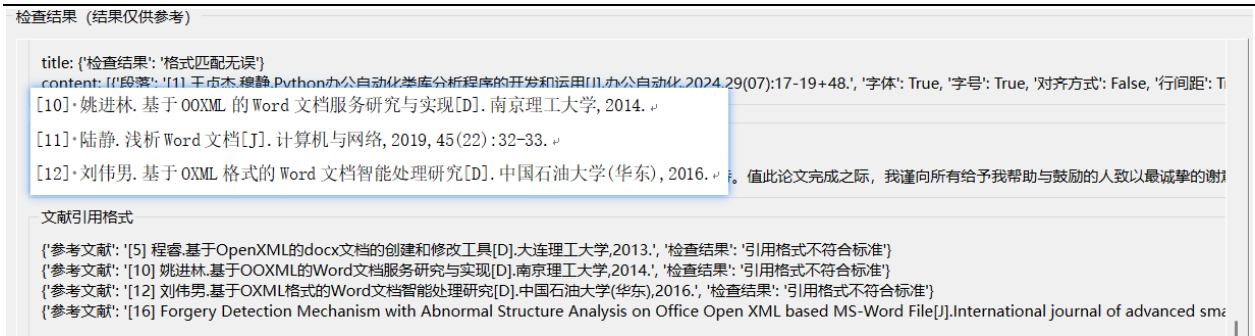


图 4.11 文献引用格式检查结果示例

④文献是学位论文时，书写格式：、

•[序号]•姓名. •题目 [D]. •授予单位所在地：授予单位，授予年.、

⑤文献是专利时，书写格式：、

•[序号]•申请人. •专利名. •国名，专利文献种类专利号[P]. 日期.、

⑥文献是技术标准时，书写格式：、

•[序号]•发布单位. •技术标准代号. •技术标准名称[S]. •出版地：

出版者，出版年.、

图 4.12 学位论文引用格式规范

该模块的格式检查与文献引用格式检查流程如图 4.13 所示：

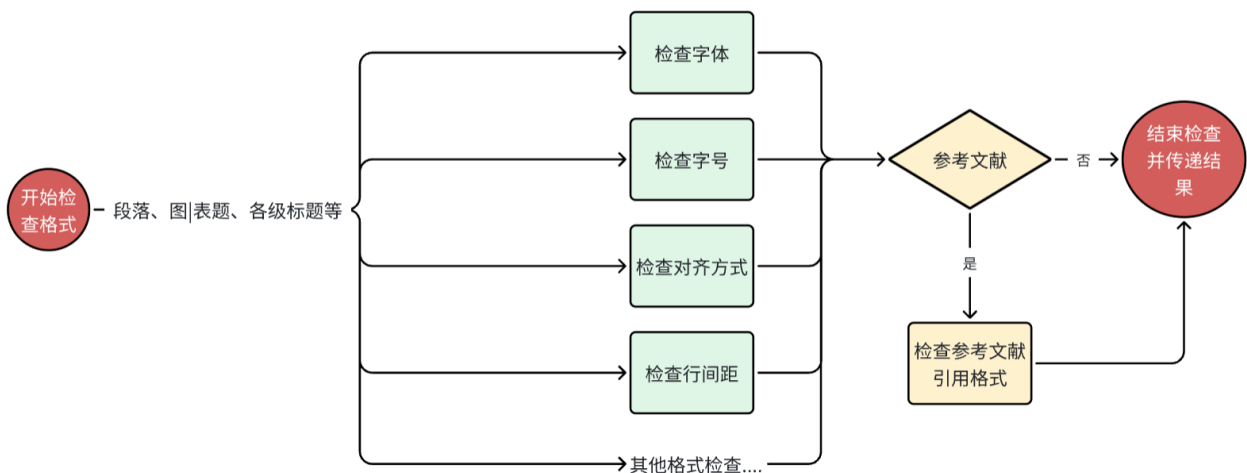


图 4.13 格式检查模块流程图

4.4.3 排版顺序检查功能

排版顺序检查功能着重审查论文各部分是否缺失以及排列顺序是否符合逻辑和规范要求。

该功能采取的实现方式为文档解析模块的简化版，同样是采用全连接无环有限状态机模型搭配关键词识别算法来确定文档的不同部分，但遍历时只识别并记录该部分，而不进行具体内容的解析。

采用省略了解析行为的全连接无环有限状态机模型搭配关键词识别算法的排版顺序检查功能流程如图 4.14 所示：

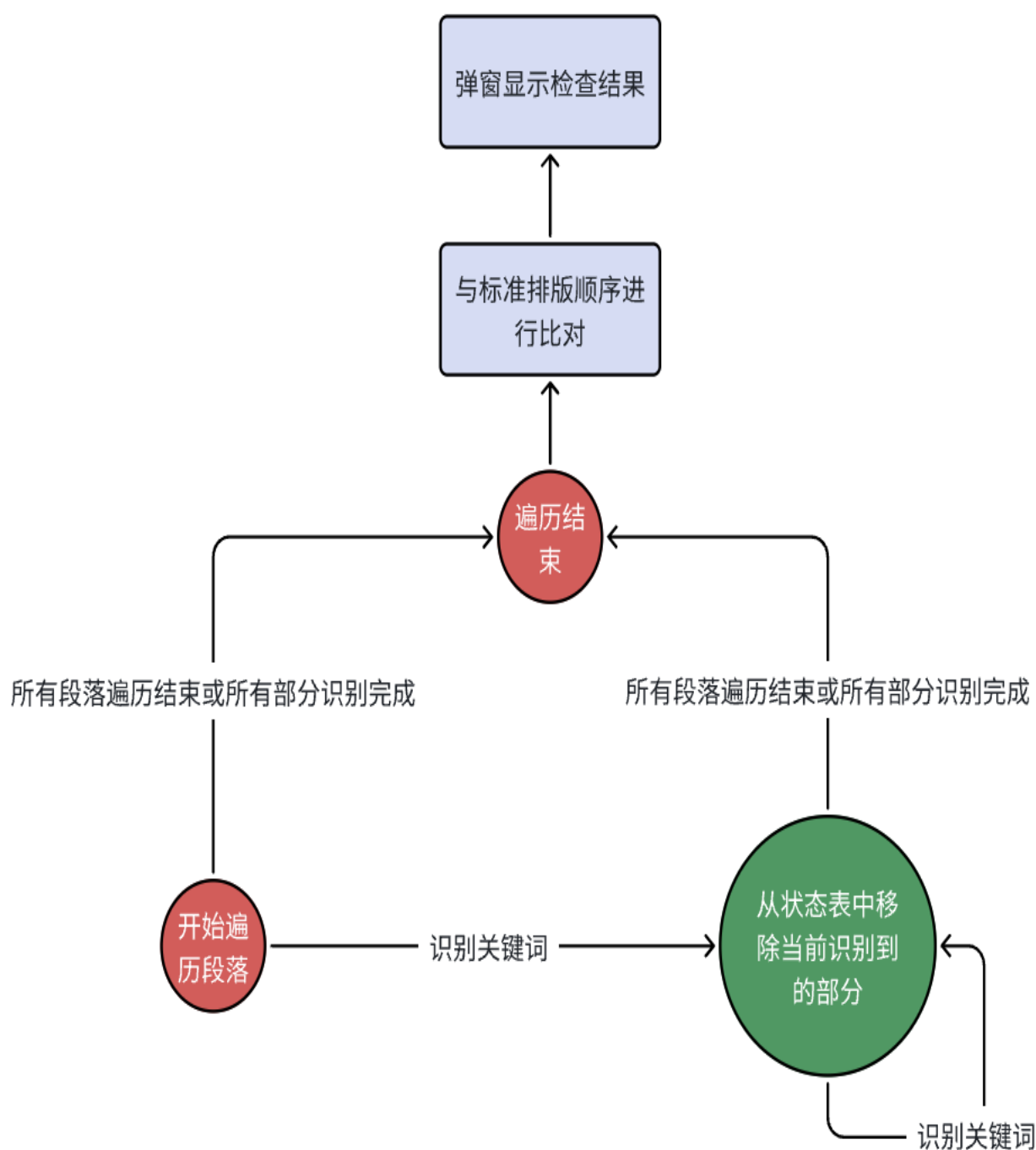


图 4.14 排版顺序检查功能流程图

遍历结束后跟标准顺序进行比对并通过弹窗来告知用户可能缺失的部分与错误的排版顺序。例如：“摘要”通常放在“目录”前，如果“摘要”出现在了“目录”后，检查就会失败，并提示用户“可能的排版错误：‘目录’排在‘摘要’前”。

如图 4.15 所示，缺失中文摘要的论文进行排版顺序检查时，会有弹窗提示缺失部分。

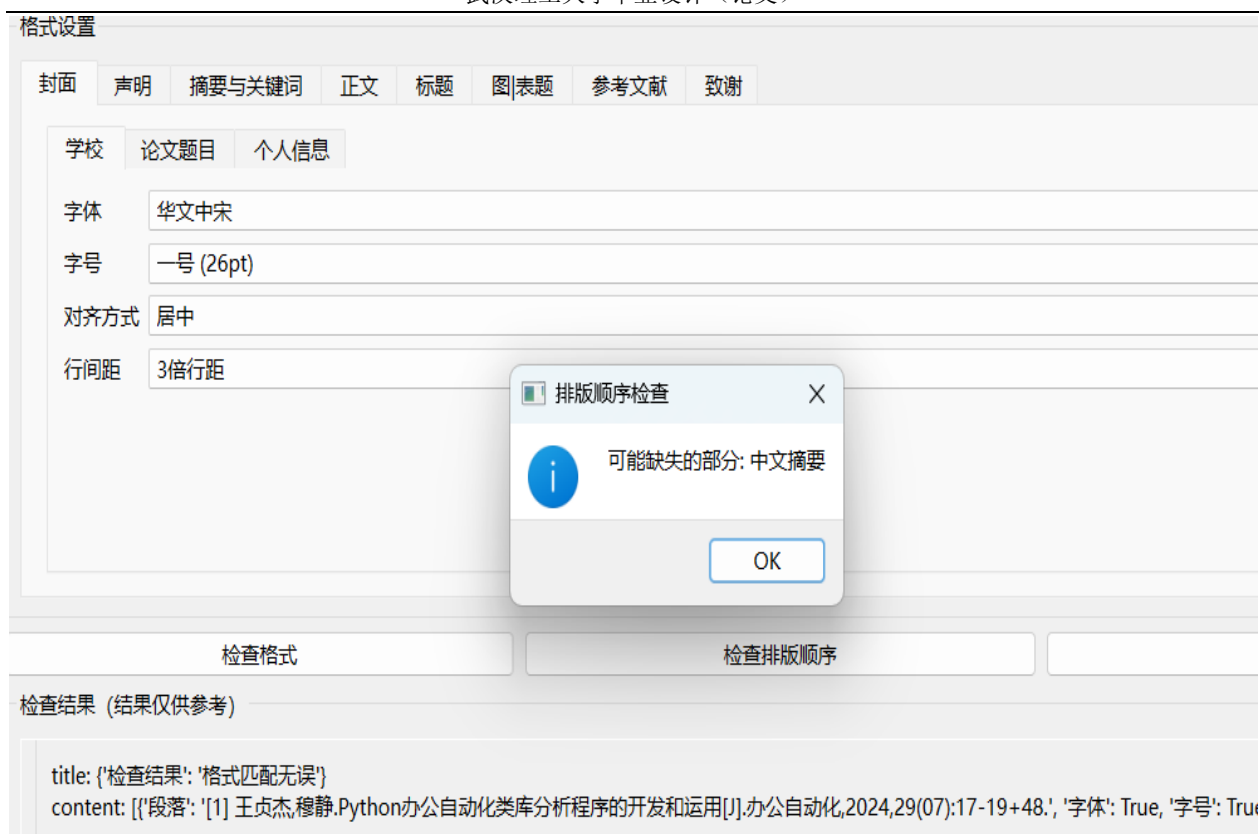


图 4.15 排版顺序检查弹窗示例

以上为本模块的核心功能介绍。性能上，模块为了提高检查效率和准确性，引入了哈希表来存储规则集，利用哈希查找的高效性，快速定位和匹配相应规则。

格式检查模块在整个系统中起着质量把控的关键作用，通过严格的规则匹配，为用户提供详细且准确的文档格式问题报告，为用户进行格式修改提供清晰指引。

4.5 格式修改模块的实现

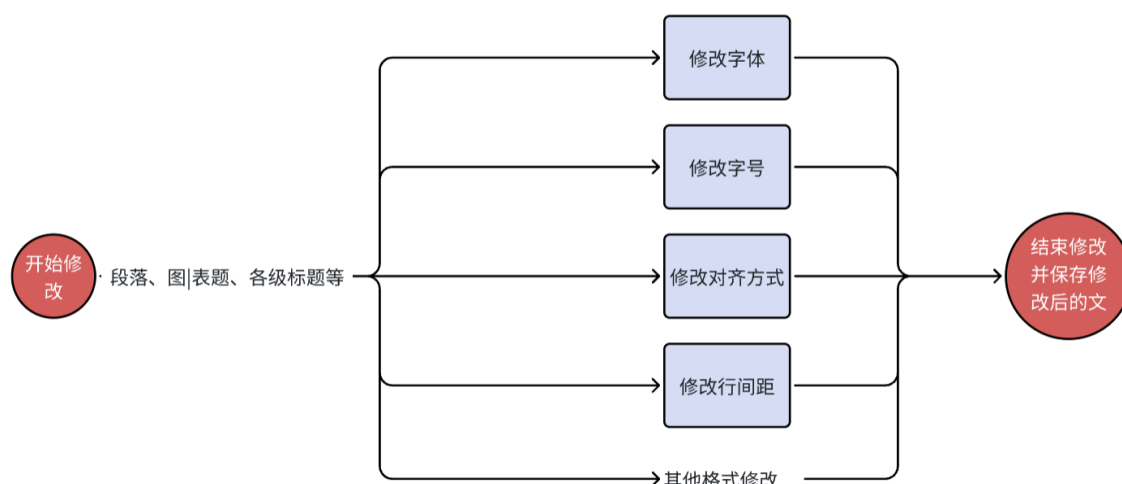
格式修改模块的主要任务是依据用户设定或默认的格式规范，对文档进行自动化的格式调整，使文档最终符合预设的格式要求，同时对修改后的文档进行保存。

4.5.1 格式修改功能

格式修改功能借助 `python-docx` 库提供的丰富 API 来实现具体操作。其核心在于能够精准对文档中的具体格式元素按照正确的格式规范进行修改。例如，通过 `paragraph.font.name` 修改字体，利用 `paragraph.paragraph_format.alignment` 设置对齐方式，借助 `WD_LINE_SPACING` 枚举值与 `line_spacing` 属性调整行距。实现思路，类似格式检查模块，按照文档的不同部分（如封面、摘要、正文等）进行格式修改。然后，针对每一类错误（如字体错误、字号错误等），利用 `python-docx` 库分别编写相应的修改逻辑。

辑。将预设规范一一应用到程序中的临时文档对象上，等待保存功能对其进行保存。

该功能的格式修改流程如图 4.16 所示：



4.5.2 修改后文档自动保存功能

此功能是要把经过格式修改的文档妥善存放起来，在方便用户查找翻阅的情况下，保证原文件不被影响，它的主要步骤包括获取保存路径以及保存修改后的文档，

获取保存路径是借助获取传入文档的原始路径，把同级文件夹目录当作修改后文档的保存路径，具体的做法是提取原文件名，在其前面加上“formatted_”这个前缀，形成新的文件名，要是原文件名为“论文.docx”，那么新文件名为“formatted_论文.docx”，并且与原始路径组合成完整的保存路径。

文档保存时，使用 `doc.save(f"formatted_{original_name}")` 生成新文件，将修改后的文档对象保存到上述生成的新路径下，并通过路径有效性校验避免覆盖原始文件。如图 4.17 所示：



以上便是格式修改模块的核心功能介绍，格式修改模块凭借严谨的格式修改流程以及安全可靠的保存机制，达成了文档格式的规范化处理以及修改后文档的妥善存储，为用户提供了符合格式要求的高质量文档。

4.6 本章小结

在这一章节当中，对借助 python - docx 库来构建的 Word 文档格式检查与修改软件各核心功能的具体实现进程展开了详尽的说明，该软件所涉及的关键模块包括系统架构设计、用户交互界面、文档解析以及格式检查与修改等方面，凭借运用模块化开发以及技术整合的方式，保障了软件可以较高的效率以及精准度去完成文档格式处理的任务。

第5章 系统测试

在项目正式开展部署工作以前，系统测试有着十分关键的作用，每一个系统都应当经历严格的测试流程，以此来防止后期开发以及运行过程中出现错误，本章会着重介绍利用 python-docx 的 Word 文档格式检查与修改软件的系统测试情况，主要运用黑盒测试方法，来保证系统功能以及稳定性。

5.1 测试环境

5.1.1 硬件环境

配备 Windows11 操作系统的笔记本电脑，处理器为 11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz 2.30 GHz；机带为 RAM 16.0 GB (15.6 GB 可用)；系统类型为 64 位操作系统，基于 x64 的处理器

5.1.2 软件环境

开发在 VSCode 1.100.2 进行，使用版本为 3.13.2 的 Python 解释器，并利用可解析和修改 Word 文档的 python-docx 1.1.2 与提供跨平台的图形界面组件与信号槽机制 PyQt6 6.9.0 两个核心库进行软件构建。

5.1.3 测试方式

人工设计与编写测试用例，执行并观察结果。

5.2 系统功能测试

本节将对系统主要功能分别进行测试，测试的主要功能包括：文档导入、格式设置、格式检查、排版顺序检查、参考文献引用格式检查、格式修改等功能。

5.2.1 文档导入功能测试

文档导入功能的测试内容主要有俩个：一个是导入不同类型文件能否成功；另一个是拖拽或手动选择文件的不同导入方式能否成功。文档导入功能的测试用例表如表 5.1 所示。

表 5.1 文档导入功能测试用例表

编号	测试项目	测试用例	预期结果	实际结果	测试结果
1	文件类型 / 导入方式	拖拽.docx 文件至软件界面	成功识别文档并显示文件路径	成功识别文档并显示文件路径	预期结果一致 操作稳定

2	文件类型 / 导入方式	拖拽.png 文件至软件界面	拒绝接受该文 件	拒绝接受该文 件	预期结果一致 操作稳定
	文件类型 / 导入方式	通过按钮打开文件选择器	文件选择器仅 显示.docx 文件	文件选择器仅 显示.docx 文件	预期结果一致 操作稳定

具体测试情形如下：

拖拽.png 文件至软件界面时拒绝接受测试如图 5.1 所示：

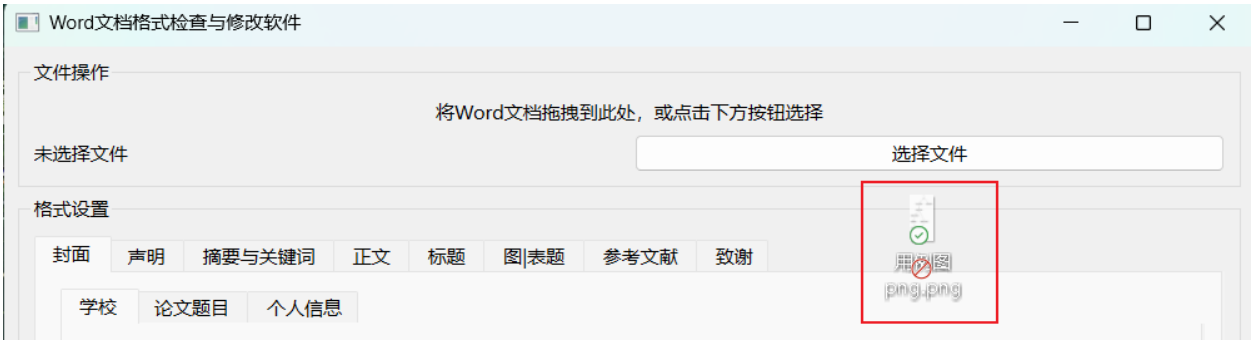


图 5.1 拖拽导入非.docx 文件

通过按钮选择文件时仅显示.docx 文件测试如图 5.2 所示：

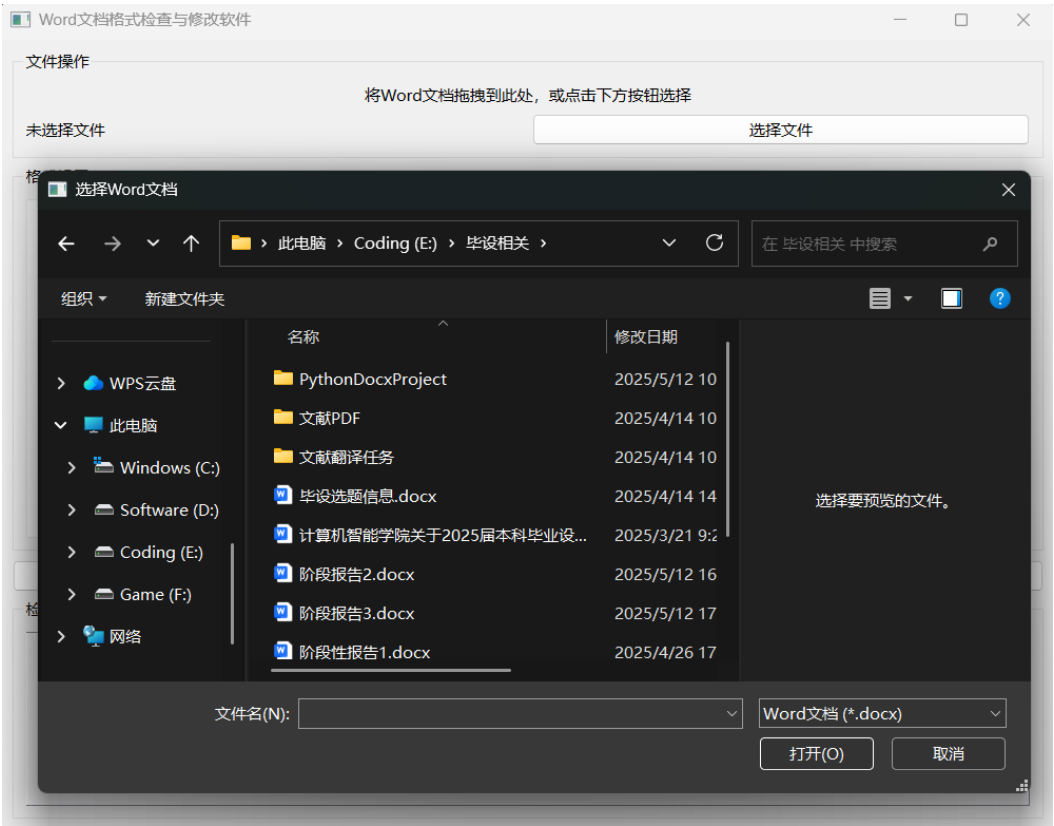


图 5.2 通过按钮选择.docx 文件

导入成功时显示文件路径测试如图 5.3 所示：

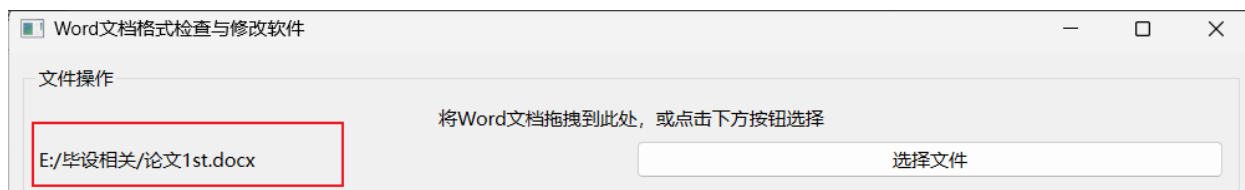


图 5.3 导入.docx 文件成功

测试结果表明，软件能够准确识别文件类型，保障后续处理的可靠性。

5.2.2 格式设置功能测试

格式设置模块给予用户预先设定各文档部分格式规范的权限，此为软件达成自动化检查与修改的根基所在，测试着重对参数保存、应用以及重置功能给予验证：比如说，当在封面模块把学校的格式设置成“华文中宋 + 一号字 + 居中 + 三倍行距”之后，参数可正保证存且在界面上显示出来。该模块保障用户可以灵活地去定义格式规则，为后续的检查与修改提供确切的依据。

设置如图 5.4 所示：



图 5.4 格式设置测试示例

测试结果表明，所有格式设置均能正常保存并显示在界面上，为格式检查与格式修改功能奠定了基础。

5.2.3 格式检查模块与结果显示功能测试

格式检查模块是软件的核心功能之一，依赖格式设置功能，需依据用户预设的规范格式进行覆盖每个部分的字体、字号、行距以及参考文献引用格式等多维度规则的检查。主要需要测试的功能有各部分的格式、排版顺序的检查以及参考文献引用格式的检查。

结果显示功能则是格式检查功能给用户的反馈窗口。

测试用例如表 5.2 所示：

表 5.2 格式检查功能测试用例表

编号	测试项目	测试用例	预期结果	实际结果	测试结果
1	格式检查	传入测试论文后点击开始检查按钮	显示各部分的具体检查结果	显示各部分的具体检查结果	预期结果一致 操作稳定
2	排版顺序检查	传入测试论文后点击检查排版顺序按钮	显示排版正确/排版疑似问题提示	显示排版正确/排版疑似问题提示	预期结果一致 操作稳定
3	参考文献引用格式检查	传入测试论文后点击开始检查按钮	检查结果底部显示格式非法的文献条目	检查结果底部显示格式非法的文献条目	预期结果一致 操作稳定

格式检查功能测试示例如图 5.5 所示：



图 5.5 格式检查测试示例

排版顺序检查测试示例如图 5.6 所示：

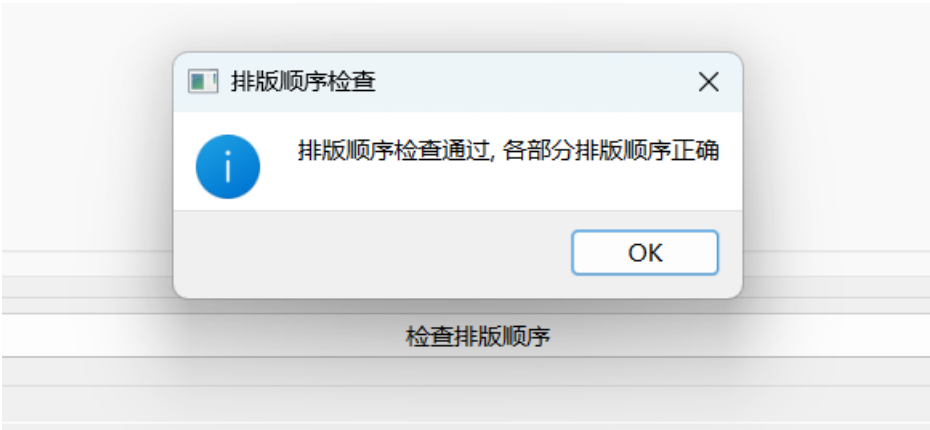


图 5.6 排版顺序检查测试示例

参考文献引用格式检查测试示例如图 5.7 所示：

检查结果 (结果仅供参考)

参考文献

title: [检查结果: '格式匹配无误']

content: [[段落: '石元伍,张怡晶.基于注意力分配的儿童教育类APP界面布局优化策略研究[J].包装工程,2023,44(16):121-131.', '字体': False, '字号': True, '对齐方式': False, '行间距': True], [段落: 'Bossard A. On Modern User Interface Development and the Case of Androi

致谢

title: [检查结果: '格式匹配无误']

content: [[段落: '总觉得来日方长，毕业遥遥无期。终于也到我扶耄于此，之前也曾看到过别人热泪盈眶的致谢，那时想我的致谢会写些什么，当我回首四年来的种种过往，思绪万千。', '字体': True, '字号': True, '对齐方式': False, '行间距': True], [段落: '我与武汉理工大学的故事

文献引用格式

[参考文献: 'Mario J N P S, Marianus M S, Nurliani M, et al. The development of android-based mathematics learning application[J]. Journal of Physics: Conference Series, 2022, 2193(1), N/A.', '检查结果: '引用格式不符合标准']

[参考文献: '高一松. 基于多样性评价的推荐算法安全检测系统设计与实现[D]. 北京邮电大学, 2023.', '检查结果: '引用格式不符合标准']

[参考文献: 'Shan L. Design and Implementation of Computer Information Security Management System in Colleges and Universities [J]. International Journal of New Developments in Education, 2023, 5 (21), N/A.', '检查结果: '引用格式不符合标准']

[参考文献: '宋琪琪. 基于Android的协同办公APP的设计与实现[D]. 北京交通大学, 2022.', '检查结果: '引用格式不符合标准']

图 5.7 参考文献引用格式检查测试示例

测试结果表明，各部分格式、排版顺序检查以及文献引用格式检查与结果显示功能表现均符合预期。

5.2.4 格式修改模块测试

格式修改模块的目标是基于检查结果实现自动化修正，同时保障数据安全。测试中，点击“确认修改”按钮后，封面的“学校”内容由原本的正确格式被自动更正为预设的格式——“宋体、五号、左对齐、0.5倍行间距”。如图 5.8 所示

格式设置

封面 声明 摘要与关键词 正文 标题 图/表题 参考文献 致谢

学校 论文题目 个人信息

字体 宋体

字号 五号 (10.5pt)

对齐方式 左对齐

行间距 0.5倍行距

检查结果 (结果仅供参考)

格式修改成功

修改后的文件: formatted_论文1st.docx

保存位置: E:/毕设相关

图 5.8 格式修改模块测试示例

修改后的文档以“formatted_原文件名.docx”另存在传入文件所在的目录下，原文件不受影响，保证了数据安全；对比新旧文档发现，格式错误完全纠正且内容无遗漏。

修改后的各项格式属性值如图 5.9 所示：

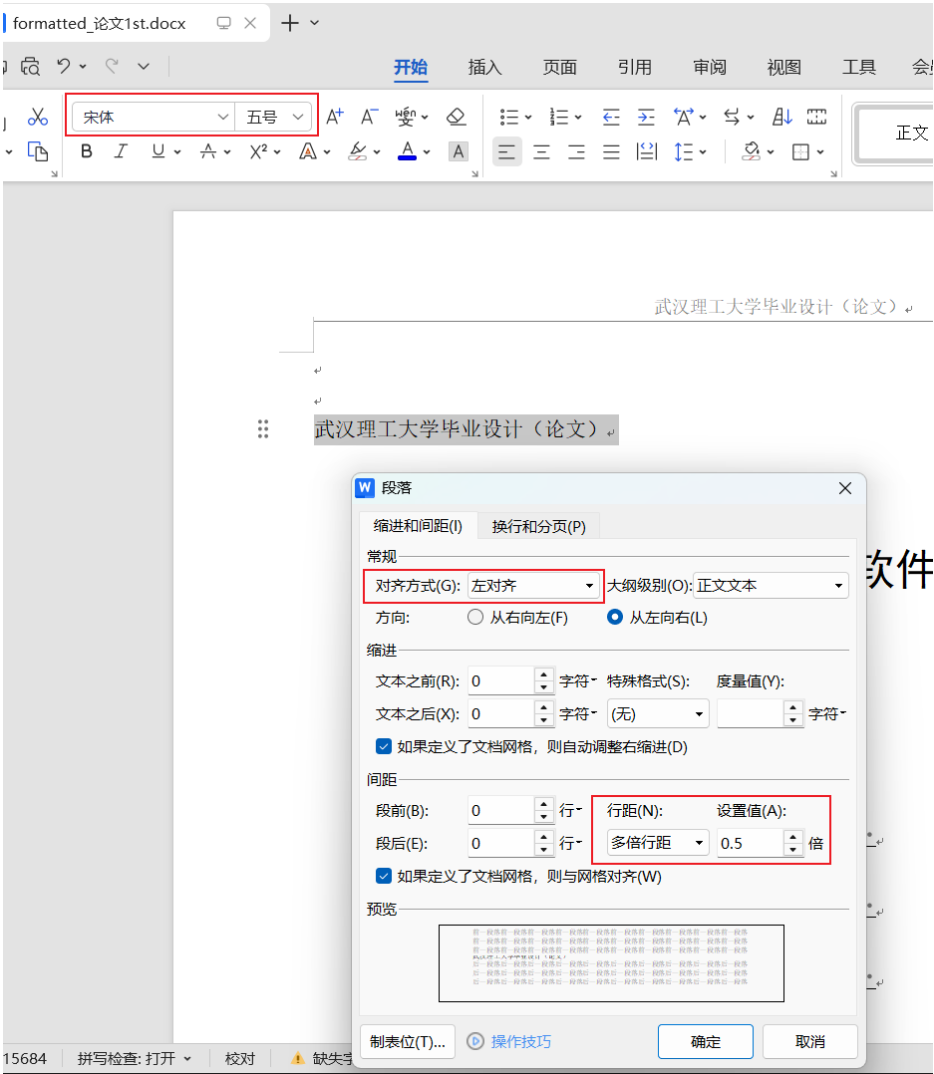


图 5.9 格式修改模块修改成功

测试结果符合预期，测试验证了自动化修改的准确性，确保用户可按预设规范修正文档格式。

5.3 本章小结

本章围绕 Word 文档格式检查与修改软件的系统测试展开，通过构建真实场景的测试环境，设计覆盖核心功能、边界条件的测试用例，全面验证了软件的可用性与可靠性。

第6章 总结与展望

6.1 总结

本研究聚焦于基于 python-docx 库的 Word 文档格式检查与修改软件的设计和实现，希望能够解决学术写作以及办公场景里 Word 文档格式规范性这一问题，经过对文档格式需求展开系统分析，设计自动化检查与修改算法，开发用户交互界面并且进行全面测试，最终达成了一款高效且精准的文档格式处理工具。

在技术实现方面，软件以 Python 编程语言和 python-docx 库为基础，结合 PyQt6 构建用户界面，借助 VSCode 开发环境来实现代码编写与调试，核心功能模块涉及文档解析、格式检查、格式修改以及用户交互，文档解析模块依靠全连接无环有限状态机模型搭配关键词识别算法，保证封面、摘要、正文等文档部分可被准确识别，格式检查模块依据自定义规则集合，运用哈希表存储规则，达成字体、字号、段落格式等多维度的快速匹配以及错误定位，格式修改模块借助 python-docx 库 API，依照用户预设规范自动化修正格式错误，保证文档符合标准。

经过系统测试验证，软件可以准确检查并纠正 Word 文档中封面、声明、摘要、正文、参考文献等部分的格式错误，支持常见文献引用格式的校验，而且操作流程稳定可靠，在实际应用中，软件提升了文档格式处理效率，减少了人工操作误差，在学术论文排版场景中，有效解决了传统手动处理的低效与高错问题。

6.2 展望

虽然研究完成的软件达成了预期的成果，不过在未来以及可以从技术深化、应用拓展以及体验优化这三个方面做的提高，在技术方面，要开发可兼容多种格式的跨平台版本，并且搭建开放的插件生态来支持自定义规则，在应用方面，可对接高校的论文管理系统以及企业的办公平台，拓展教育和企业级的场景，增添多语言支持以适应国际化的需求。在用户体验方面，会优化可视化格式对比工具以及错误提示信息，加强数据加密和隐私保护机制，保证操作的便捷性与安全性，依靠对以上这些方面的优化，软件有机会成为智能化、全场景的文档格式处理通用工具，推动办公自动化水平获得的提升。

参考文献

- [1] 王贞杰, 穆静. Python 办公自动化类库分析程序的开发和运用[J]. 办公自动化, 2024, 29(07):17-19+48.
- [1] 丁烨敏. 基于 Python+Open XML 的毕业设计说明书格式自动检测系统[J]. 科学技术创新, 2023, (20):121-124.
- [3] 罗文兵, 王健, 周月. 多 DOCX 文档文本内容批量替换工具的实现和应用[J]. 电子技术与软件工程, 2022, (06):56-60. DOI:10.20109/j.cnki. etse. 2022. 06. 014.
- [4] 李秀贤. 基于 Python 的学位论文自动排版研究[J]. 现代职业教育, 2020, (09):128-129.
- [5] 程睿. 基于 OpenXML 的 docx 文档的创建和修改工具[D]. 大连:大连理工大学, 2013.
- [6] 林晓, 吴为民, 刘勇峰. 基于 Open XML 和有限自动机的试卷自动生成系统[J]. 顺德职业技术学院学报, 2023, 21(03):38-43.
- [7] 纳利军. 基于 Open XML 的毕业论文格式修订系统的设计与实现[J]. 电脑知识与技术, 2022, 18(09):41-43. DOI:10.14004/j.cnki. ckt. 2022. 0564.
- [8] 秦志红. DOCX 文档解析及隐藏信息提取算法[J]. 通信技术, 2021, 54(11):2557-2563.
- [9] 吴为民. OXML 结构化格式的 Word 试卷智能处理系统[J]. 现代计算机, 2020, (34):105-107.
- [10] 姚进林. 基于 OXML 的 Word 文档服务研究与实现[D]. 南京:南京理工大学, 2014.
- [11] 陆静. 浅析 Word 文档[J]. 计算机与网络, 2019, 45(22):32-33.
- [12] 刘伟男. 基于 OXML 格式的 Word 文档智能处理研究[D]. 中国石油大学:中国石油大学(华东), 2016.
- [13] Machlis S. How to create Word docs from R or Python with Quarto [J]. InfoWorld.com, 2022. <https://www.infoworld.com/article/2336531/how-to-create-word-docs-from-r-or-python-with-quarto.html>.
- [14] Asako O. Could Authors of Academic Reports be Discerned Using Formatting Information Obtained by Parsing XML of .docx Documents? [J]. 電気学会論文誌 C (電子・情報・システム部門誌) / 電気学会論文誌, 2023, 143 (1): 91-100.
- [15] Khan F, Chen B, Varro D, et al. An Empirical Study of Type-Related Defects in Python Projects[J]. IEEE Transactions on Software Engineering, 2022, 48(8):3145-3158.
- [17] Lee H, Lee H W. Forgery Detection Mechanism with Abnormal Structure Analysis on Office Open XML based MS-Word File[J]. International journal of advanced smart convergence, 2019, 8(4):47-57.

致 谢

时光荏苒，岁月如歌。从论文选题到最终定稿，这段旅程凝聚了太多人的心血与支持。值此论文完成之际，我谨向所有给予我帮助与鼓励的人致以最诚挚的谢意。

首先，衷心感谢我的指导老师孙玉芬教授。从软件功能设计到代码实现，从论文框架搭建到细节打磨，孙老师始终以严谨的治学态度、渊博的专业知识和耐心的指导给予我悉心关怀。她多次在我遇到技术瓶颈时悉心点拨，帮助我理清思路、优化方案，更以精益求精的学术精神感染着我，让我深刻体会到科研工作的责任与乐趣。孙老师的教诲与鼓励，将成为我未来学术与职业道路上的宝贵财富。

感谢武汉理工大学计算机与人工智能学院的各位老师，正是你们在课堂上的悉心传授，让我具备了扎实的专业基础；在实验室中的耐心指导，让我掌握了科学的研究方法。特别感谢辅导员老师在生活与学业上的关心与支持，你们的包容与鼓励让我能够心无旁骛地投入到研究中。

感谢我的家人，你们的理解与支持是我前进的最大动力。父母多年来的辛勤付出与无条件的爱，让我能够专注于学业；兄弟姐妹的陪伴与鼓励，让我在遇到困难时始终坚信自己并不孤单。你们的温暖与包容，是我永远的港湾。

感谢并肩作战的同学们，我们在实验室中共同探讨问题、分享经验，在挫折中相互鼓励、共同成长。那些一起攻克技术难题的日夜，那些为了一个小目标而欢呼的瞬间，都将成为我青春记忆中最珍贵的片段。

最后，感谢评审老师在百忙之中审阅我的论文，感谢你们提出的宝贵意见与建议。由于本人水平有限，论文中难免存在不足，恳请各位老师批评指正。

毕业不是终点，而是新的起点。在未来的日子里，我将带着在武汉理工大学学到的知识与品格，勇敢面对挑战，努力回馈社会，不负母校的培养与期望。

衷心祝愿母校积历史之厚蕴，更展宏图，再谱华章！祝愿所有关心与帮助过我的人身体健康、工作顺利、万事胜意！